

# CABLE GUARD

by Kirchenfeldrobotics

## Guarding the lifelines of the Alps

An autonomous robot that inspects steel wire ropes



---

### Project report

World Robot Olympiad 2026, Future Innovators, Senior  
WRO Open Championship, Zagreb, October 2026

**Team Kirchenfeldrobotics**, Bern, Switzerland  
Jakob Gutersohn, Nils Buchli and Valéry Piot  
Coach: David Bänziger

# Contents

---

|          |                                     |           |
|----------|-------------------------------------|-----------|
| <b>1</b> | <b>Our team</b>                     | <b>1</b>  |
| <b>2</b> | <b>Summary of the project idea</b>  | <b>2</b>  |
|          | The problem                         | 2         |
|          | Our solution                        | 2         |
|          | What it is worth                    | 2         |
| <b>3</b> | <b>Our robotic solution</b>         | <b>3</b>  |
|          | 3.1 How the idea developed          | 3         |
|          | 3.2 What already exists             | 4         |
|          | 3.3 Construction: mechanics         | 4         |
|          | 3.4 Construction: electronics       | 7         |
|          | 3.5 Coding                          | 8         |
|          | 3.6 Challenges and tests            | 11        |
|          | 3.7 Use of AI                       | 13        |
| <b>4</b> | <b>Social impact and innovation</b> | <b>14</b> |
|          | 4.1 Who Cableguard helps            | 14        |
|          | 4.2 Possible negative effects       | 15        |
|          | 4.3 A tried use case                | 15        |
|          | 4.4 Key innovation                  | 16        |
|          | 4.5 From prototype to start-up      | 16        |
|          | 4.6 Next steps                      | 18        |
|          | <b>Sources</b>                      | <b>i</b>  |

# 1 Our team

Hi WRO, we are Kirchenfeldrobotics (kirchenfeldrobotics.ch), a registered student robotics club from Bern. All three of us are in the third year of Gymnasium Kirchenfeld.

Last year Jakob and Valéry built Farmatic, a modular vertical farm, and won 3rd place and the Team Award at the WRO Open Championship in Slovenia [1]. This season we took on something that carries lives in the Alps and is still checked by eye: the steel rope. We iterated Cableguard through three versions in one season, and on 20 June 2026 it won 2nd place at the Swiss final, which brought us to Zagreb.

We want Cableguard to outlive this competition: a robot on every historic ropeway, and every broken wire found on the day it breaks, not on the day the rope fails. Our roadmap leads from this prototype to a pilot and a company (section 4.6).



**Figure 1.** Our team with Cableguard: Jakob Guttersohn, Valéry Piot and Nils Buchli.

| Member           | Responsibility           | In this project                                                                                                                                                                                                    |
|------------------|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Jakob Guttersohn | Hardware                 | Designed every custom part in CAD, assembled the robot and soldered                                                                                                                                                |
| Nils Buchli      | Software and electronics | Robot software on the Raspberry Pi and the server. Set up and tested the electronics with Valéry                                                                                                                   |
| Valéry Piot      | Software and electronics | Web app, login, the AI model with its training and parts of the Raspberry Pi. Set up and tested the electronics with Nils and wired all components. Set up the Raspberry Pi 5 for a GitHub auto-redeploy workflow. |
| David Bänziger   | Coach                    | Helped us plan and organise and reviewed our work. He won the WRO Open Championship 2025 with his own team.                                                                                                        |

Everything around the robot was shared work: all three of us wrote sponsoring requests, built the finance plan and wrote this report. Our goal is that each of us is responsible for one part and still understands the others. Since August we have written about 9,700 lines of code in more than 150 commits and 20 merged pull requests. We trained our own neural network on about 22,000 rope images, wrote a program for the Pi's PIO hardware that times every motor step to the microsecond, and went through three hardware generations: our own ESP32 circuit board, a magnetic sensor ring with eight Hall sensors, and finally the Raspberry Pi 5. Each time we tested, changed and tested again.

## 2 Summary of the project idea

### The problem

**On 23 May 2021, shortly before the top station of the Stresa–Mottarone cable car in Italy, its haul rope snapped.** The cabin rolled back, came off at a pylon and fell into the forest. Fourteen people died, and a five-year-old boy was the only survivor [21]. Court experts later found that 68 % of the wires at the break had failed from fatigue and corrosion long before that day, and that correct checks would have revealed it [29]. The rope did not fail suddenly. It weakened over years, and this is exactly the kind of slow damage that regular inspection is meant to catch.



**Figure 2.** Mottarone, Italy, 23 May 2021: rescuers at the cabin after its haul rope broke. Press photo.

Switzerland has about 2,450 cableways [2]. 129 are listed as heritage [3], and every second small cableway has financial problems [4]. Their steel ropes age: wires break, corrosion and lightning leave marks [5]. Today a person checks a rope by riding on a slow-moving gondola, which is slow, expensive and prone to error [6]. Climate change adds pressure. In 2022 the Swiss permafrost reached record temperatures at two thirds of the measuring sites [33], and the ground under high-alpine stations and pylons is becoming unstable [34]. The check that saves lives has to become cheaper and more frequent, not rarer.

### Our solution

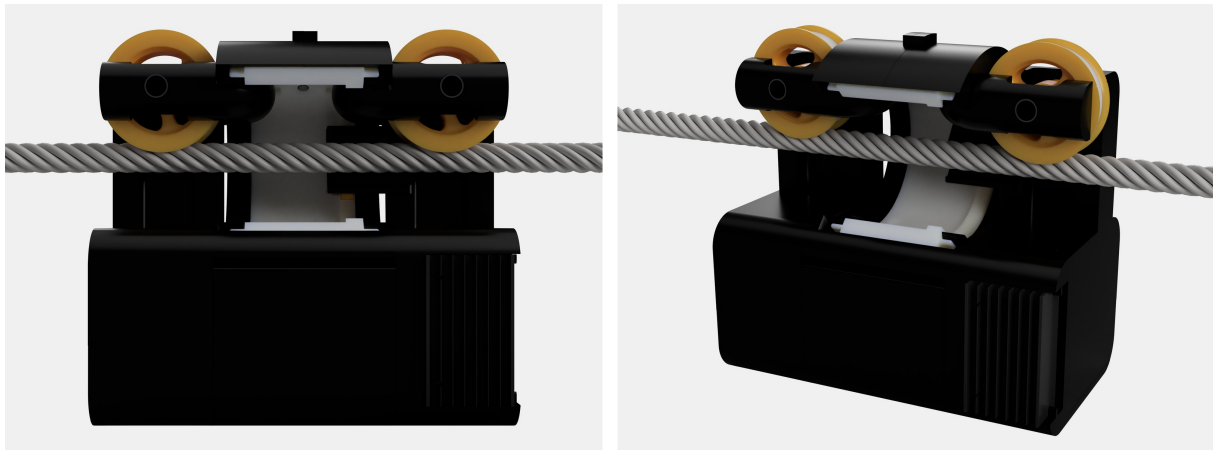
The operator places Cableguard on the rope and presses start, and the robot works alone. Two cameras on a rotating ring photograph all four sides of the rope continuously, a YOLO26 network that we trained on about 22,000 rope images marks eight classes of damage, and the steps of the motor give the position of every photo. A web app shows the session live and visualises inspections.

### What it is worth

Every finding comes with a photo and its exact position, and an operator can inspect as often as needed without a crew. Most rope damage shows on the surface. For damage inside the rope, a magnetic sensor ring can be added later (section 3.1). It covers the time between two tests, when damage like that at Mottarone grows unseen.

## 3 Our robotic solution

Cableguard is a self-driving inspection robot with two stepper motors, two cameras, a distance sensor and a Raspberry Pi 5. It weighs 3 kg and hangs on the rope by two wheels (Figure 3). This chapter describes how the idea changed, what already exists, how we built and programmed the robot and how we solved the overheating of the Raspberry Pi 5 caused by the neural network.

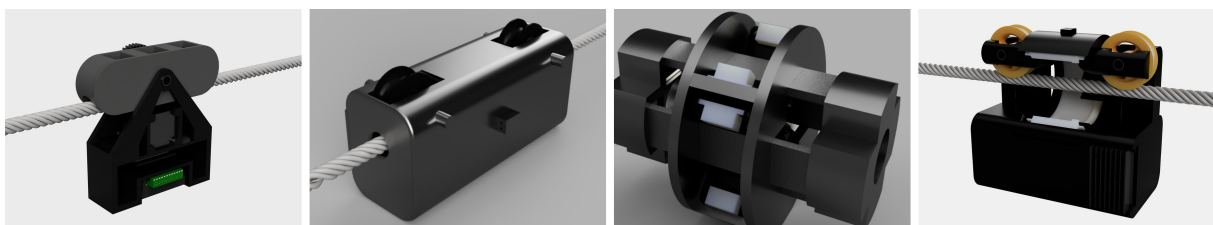


**Figure 3.** Cableguard on a rope, rendered from our CAD model.

### 3.1 How the idea developed

We have rebuilt Cableguard twice (Figure 4). Each version answered the same question in a different way: how do we find damage on a rope?

|                   | Version 1, regional final (May 2026)                                                      | Version 2, Swiss final (June 2026)                                   | Version 3, Open Championship (October 2026)                                   |
|-------------------|-------------------------------------------------------------------------------------------|----------------------------------------------------------------------|-------------------------------------------------------------------------------|
| Finds damage with | One camera, images analysed on a server with OpenCV                                       | Magnetic flux leakage: a ring of eight Hall sensors around the rope  | Two cameras on a rotating ring, YOLO26 network on the robot                   |
| Computer          | ESP32-S3 on our own PCB plus an STM32H7                                                   | ESP32-S3 on our own PCB                                              | Raspberry Pi 5                                                                |
| What we learned   | PLA wheels slip on steel. Images sent through two microcontrollers arrived too unreliably | The readings of the Hall sensors were too unreliable to judge a rope | First completely reliable solution. More about it in the rest of this chapter |



**Figure 4.** The three versions as renders from our CAD models. From left: version 1, version 2 closed, the magnetic sensor head inside version 2, and version 3.

Version 2 used magnetic flux leakage (MFL) to read the magnetic field lines with Hall sensors. MFL analyses the inside of the rope, which no camera can. We built the sensor part, a ring of eight Hall sensors that the rope passes through. Three things stopped us there. The rope has to be magnetised close to saturation by very strong magnets in an iron yoke [8], and after we reviewed this approach, we identified the main problem: such a head is hard to install “due to the

large magnetic force between the magnet and the specimen” [8]. The yoke has to be machined from iron, and we have a 3D printer and no CNC machine. And turning the sensor readings into a statement about the rope needs a model of the magnetic field, which means solving Maxwell’s equations for the rope. Our readings were too unreliable for that.

After the Swiss final we made cameras the main sensor, for three reasons. We can build a camera system ourselves. A photo is evidence that anyone can check, while a magnetic signal gives data for a trained inspector. And thousands of labelled photos of damaged ropes are public, so we could train a network on images from different environments, so that our model is not overfitted to a small dataset of our own images. A working camera robot is also the prototype we need to find partners and funding for the magnetic head. The camera is also extremely reliable and works in most cases. It is almost as reliable as magnetic flux leakage, because damage that makes a steel rope dangerous has to be big, and we can also detect deformations of the rope, so the camera indirectly shows a defect inside as well.

The magnetic idea stays in the design as an upgrade. The software sorts every finding into the two groups of magnetic testing, local faults (LF) and loss of metallic area (LMA), the same two groups in which the European rope standard EN 12927 reports a magnetic test [28], and the protocol has a message for findings from a second sensor. In August we started the software again from an empty repository, now for a Raspberry Pi 5, and wrote version 3.

### 3.2 What already exists

| Method                               | How it works                                                                         | Strength                                                                       | Limit                                                                                 |
|--------------------------------------|--------------------------------------------------------------------------------------|--------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| Visual inspection by a person        | An inspector rides on a slow-moving gondola and inspects by eye                      | Finds scratches, notches and early corrosion that the magnetic test misses [5] | Slow, depends on the inspector, leaves no photo [6]                                   |
| Magnetic rope testing                | Magnets magnetise the rope, sensors measure the field that leaks out at a defect [8] | Finds broken wires inside the rope [5]                                         | Needs a testing body. The result is a curve without a picture and extremely expensive |
| Camera systems of testing institutes | Four cameras around the rope, a measuring wheel gives the position [9]               | Photos as evidence, software measures diameter and lay length [5]              | Expert equipment that a testing body brings and operates and extremely expensive      |
| Cableguard                           | Drives along the rope with two cameras on a rotating ring and a network on board     | Works alone, photo and position for every finding, compares inspections        | Sees the surface. But with our network we can see inside indirectly (section 3.1)     |

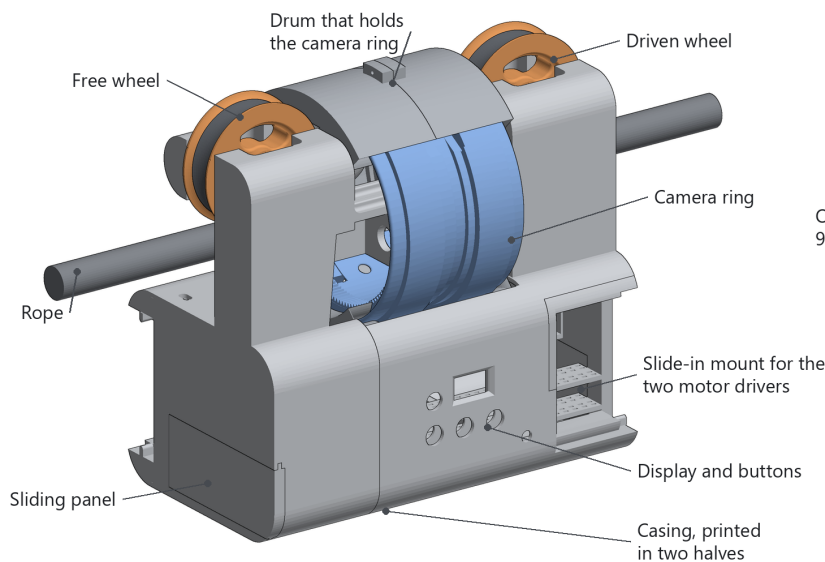
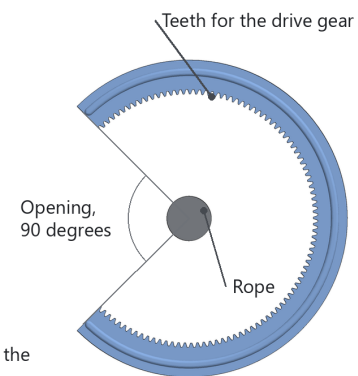
A magnetic signal is “only a 1D image of the real damage”, every doubtful signal has to be checked by eye, and the rope ends are not tested magnetically at all [27]. That confirmed our plan: cameras first, the magnetic head later, both on the same robot. Cableguard differs from the systems above in three points. It is meant to belong to the operator, it uses two cameras and a motor in place of four cameras, and its network decides how fast it drives.

### 3.3 Construction: mechanics

The body of the robot hangs below the rope. Two wheels on top carry it, and the rope passes through the upper part of the casing, where a ring with the two cameras surrounds it (Figure 5).



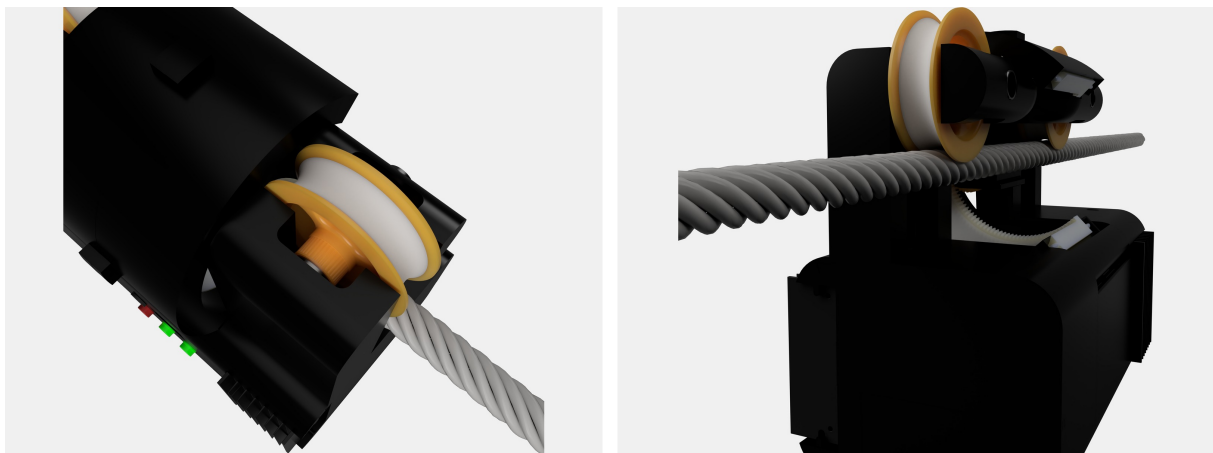
| Property           | Value                                            |
|--------------------|--------------------------------------------------|
| Size of the casing | 277 x 160 x 245 mm                               |
| Weight             | 3 kg                                             |
| Visualisation rope | Steel rope, 22 mm diameter, 1.6 m long           |
| Scan speed         | About 3 cm per second, calculated in section 3.5 |
| Power              | 3S LiPo battery                                  |

**The printed parts in place, on a rope of 22 mm****The camera ring, seen along the rope**

**Figure 5.** The printed parts of the robot and the camera ring, drawn from our CAD model. Motors, belt, cameras and electronics are not shown.

## Drive

The robot rides on two wheels. A NEMA 23 stepper motor drives one of them through a GT2 belt with a printed tensioner, and the second wheel runs free. The first wheels were printed from PLA and slipped on the steel. The current wheels have an insert of flexible TPU with a V-shaped groove (Figure 6). The weight of the robot presses the rope into the V, and this wedge gives the grip that a flat wheel does not have.



**Figure 6.** A wheel on the rope, seen from two sides.

We chose a stepper motor because it moves in fixed steps, 200 per turn, each split into 8 microsteps by the driver. The robot counts every microstep it sends, so it knows its position without an extra sensor, and one step is a fraction of a millimetre on the rope. The weakness of counting

steps is slip: if the wheel slips, the robot believes it is further along the rope than it is. We have not measured slip yet, but we have a solution for it: later, the robot could correct this error itself by comparing its own photos and GPS position, without any marks on the rope. We decided not to use GPS for the position itself, because it is only precise to roughly 3 to 10 m, which is not as precise as the steps of the robot, so we use steps. However, GPS is planned for a future pilot to calculate the error of the step count.

### Why a camera ring

A rope is round, and one camera sees less than half of it. Near the edge of the picture the surface curves away and the wires become too narrow to judge. The testing institutes therefore mount four cameras around the rope [9].

We wanted the same coverage with fewer parts. The Raspberry Pi 5 has two camera connectors [11], and every further camera adds a cable, weight and a picture for the network to process. So two cameras sit opposite each other on a ring, and a second NEMA 23 turns the ring through a printed gear (Figure 7). The cameras take a pair of photos, the ring swings them 90 degrees, and they take a second pair of the two sides that were hidden.



**Figure 7.** The camera ring (white) with its inner teeth and the gear that turns it. Right: the same unit from the side with its motor.

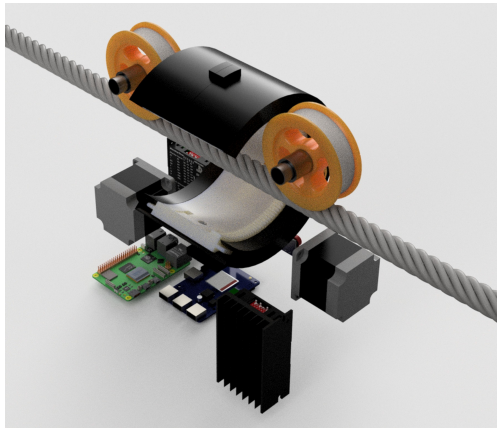
The ring has an opening of 90 degrees. To put the robot on a rope, the ring turns to its open position, the rope slips through the opening into the centre of the ring and the wheels come to rest on the rope. Two rules follow from the hardware. The ring always swings back the way it came, because the cameras hang on their cables and a ring that kept turning would wind them up. And since the ring has no end switch, whoever sets up the robot leaves it at its parking angle and the software counts every step from there.

### Casing and material

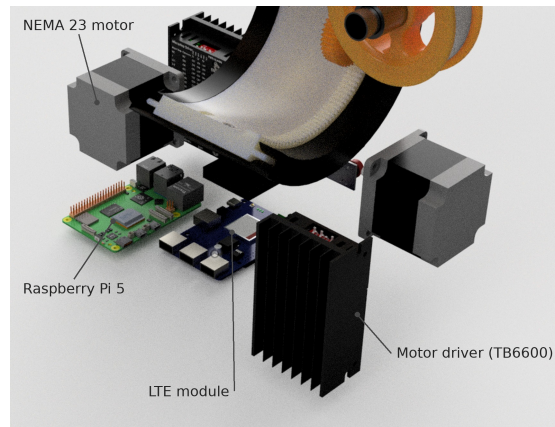
Every structural part is 3D printed in PLA: both halves of the casing, the drum that holds the camera ring, the wheel hubs and all mounts, 16 pieces in total. Only the axles, spacers, screws and belts are bought parts. We print because we can redesign a part ourselves and test it a day later, without a workshop.

The casing is a prototype. It encloses the electronics (Figure 8), but it has no seals, and we have not tested it in rain or frost yet. PLA also softens at about 57 °C [12], which a dark casing can reach in the summer sun. For a robot in weekly use we plan a casing of ASA or aluminium with sealed openings (section 4.6). The design stays the same for that step. Only the material and the seals change.





**Figure 8.** Inside Cableguard with the casing hidden. The rope runs through the drum with the camera ring, the motors and the electronics hang below it.



**Figure 9.** The electronics below the camera ring. Each motor has its own TB6600 driver.

### 3.4 Construction: electronics

One Raspberry Pi 5 runs everything on the robot (Figure 9): both motors, both cameras, the neural network and the connection to the server. In version 1 this work was split between two microcontrollers that had to talk to each other over a serial cable.

| Component          | Part                                                                                         | Connected to                                                                    |
|--------------------|----------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| Computer           | Raspberry Pi 5                                                                               |                                                                                 |
| Drive motor        | NEMA 23 stepper motor with TB6600 driver                                                     | Step GPIO 18, direction GPIO 23                                                 |
| Ring motor         | NEMA 23 stepper motor with TB6600 driver                                                     | Step GPIO 12, direction GPIO 16                                                 |
| Cameras            | 2 x Raspberry Pi Camera Module 3 Wide with LED light                                         | Both camera ports of the Pi                                                     |
| Distance sensor    | TOF200C (VL53L0X laser sensor)                                                               | I2C bus                                                                         |
| Display            | 0.96 inch OLED, 128 x 64 pixels                                                              | I2C bus                                                                         |
| Buttons and buzzer | Red start button, white stop button, buzzer. A third button is wired and has no function yet | GPIO 5, 6 and 26                                                                |
| Mobile link        | LTE module with SIM card                                                                     |                                                                                 |
| Power              | 3S LiPo battery (11.1 V nominal) and step-down converter                                     | Battery voltage directly to the motor drivers, 5 V from the converter to the Pi |

Three decisions shaped the wiring.

**One computer.** A neural network needs a real processor, and the Pi 5 has the two camera ports and the GPIO pins we need [11]. A photo, the motor position at that moment and the result of the network now live in one program, and nothing has to be synchronised over a cable.

**Usable without a phone.** Three buttons, a buzzer and a small display are enough to start a run, stop it and see what the robot is doing. Each state has its own sound, and the display shows connection, position and speed. The Pi pulls both button pins high from the moment it boots, so a button held down during start-up cannot start the motor.

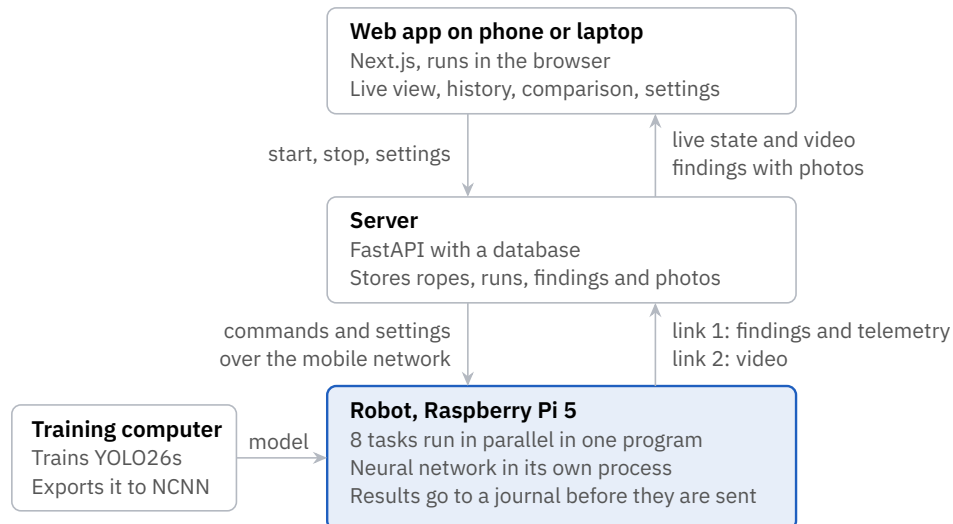
**Cables that explain themselves.** Three voltage levels, 12 V, 5 V and 3.3 V, meet in a small space. Every cable carries a three-digit label with two digits for the component and one for the pin. 013 is pin 3 of the drive motor driver, its step input. One file in the repository maps every label to a pin of the Pi.

The sensor ring of version 2 with its eight Hall sensors is not on the robot. We kept it, and section 4.6 describes how the magnetic head could come back.

### 3.5 Coding

The software runs on three devices. The robot inspects on its own, the server stores every finding and the web app is the window for the operator (Figure 10). We wrote in Python, TypeScript and C. Everything else we use is open source and free for everyone: FastAPI, Next.js, Ultralytics YOLO, NCNN and the Raspberry Pi libraries for the cameras and the PIO block.

**The robot inspects alone, the server stores, the web app shows**



**Figure 10.** Software architecture: three devices and two links from the robot to the server.

#### One scan cycle

1. Both cameras take a photo while the ring stands still.
2. The ring swings 90 degrees. During the swing the on-board AI model analyses the two photos.
3. Both cameras take a second photo of the two sides that were hidden.
4. The ring swings back and the network analyses the second pair.
5. Every result is written to a journal with the position of the robot at the moment of the photo.

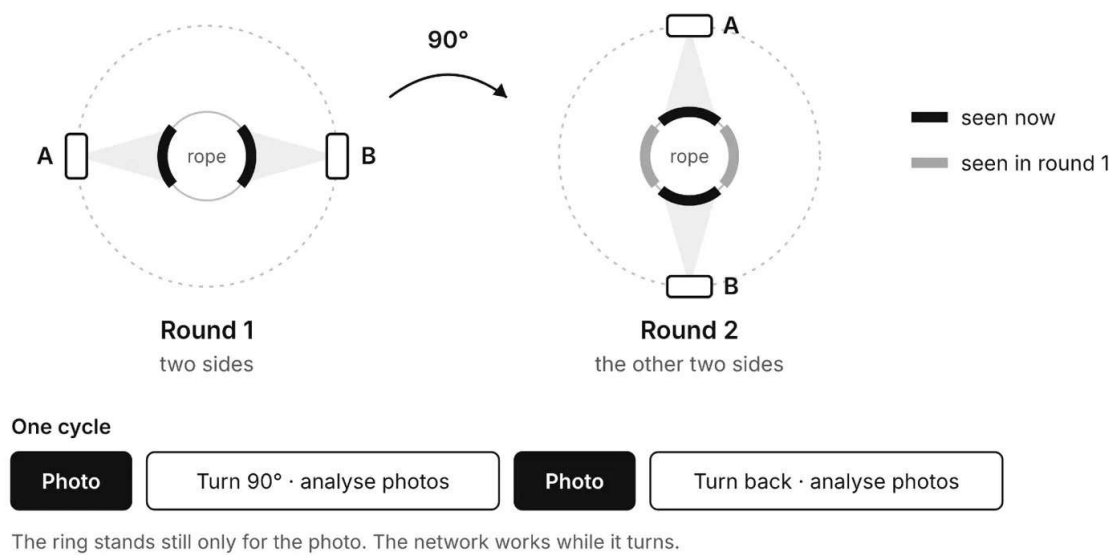
Only the shutter needs a ring that stands still. The network can work while the cameras move, so turning and analysing happen at the same time (Figure 11).

#### The robot sets its own speed

In the first weeks the operator chose the speed in the web app. A robot that drives faster than its network can analyse leaves stripes of rope that nobody looks at, so now the robot measures itself. At every start it runs nine test cycles and times the cameras and the network separately. One photo covers a length  $L$  of rope, and the robot may travel exactly  $L$  per cycle.

| What the robot calculates | Formula                                                                   |
|---------------------------|---------------------------------------------------------------------------|
| Time for one quarter turn | $t_{\text{turn}} = \max\left(\frac{H \cdot D}{R}, t_{\text{ring}}\right)$ |
| Time for one cycle        | $T = R \cdot \left(\frac{C}{R} + t_{\text{turn}}\right)$                  |
| Driving speed             | $v = \frac{L}{T}$                                                         |

$C$  is the camera time and  $D$  the network time of one cycle,  $R = 2$  the number of photo rounds and  $t_{\text{ring}}$  the fastest quarter turn of the ring.  $H = 1.5$  is a safety factor, because the network needs longer



**Figure 11.** Scan cycle of the camera ring: two rounds of photos and one quarter turn.

for a photo with damage than for the clean rope it was timed on. An example: with  $L = 60$  mm,  $C = 0.14$  s and  $D = 1.2$  s, which is 0.3 s per photo, the cycle takes  $T = 1.94$  s and the robot drives 31 mm per second, about 110 m per hour. The operator only chooses start, stop and the direction. But we can easily scale this up with a more capable computer like an NVIDIA Jetson Orin Nano (roughly CHF 300), which has a GPU optimised for matrix computations and runs the AI model faster.

### Knowing the position

A finding is only useful with its position, and the position is the count of motor steps. Every pulse has to arrive on time, also while the processor is busy with the network. Our first driver made the pulses in Python, and the motor stuttered whenever the network ran.

The Raspberry Pi 5 contains a hardware block called PIO, four small state machines that run independently of the processor [13]. We wrote a program of seven instructions for it and a small C library around it. The software hands the hardware a block of pulses, and the hardware plays it with a resolution of one microsecond. The robot adds a block to its position only after the hardware reports it as played, so the count cannot run ahead of the motor.

```
.side_set 1 opt
.wrap_target
    pull block    side 0    ; wait for a block, STEP stays low
    out x, 16      ; x = steps - 1, OSR keeps the low-time count
step:
    mov y, osr    side 1 [7] ; STEP high for 10 cycles
    nop          [1]
low:
    jmp y-- low   side 0    ; STEP low for y + 1 cycles
    jmp x-- step
    push noblock      ; block done
.wrap
```

The complete program of the PIO block for one stepper motor, from robot/src/motion/stepgen.c.

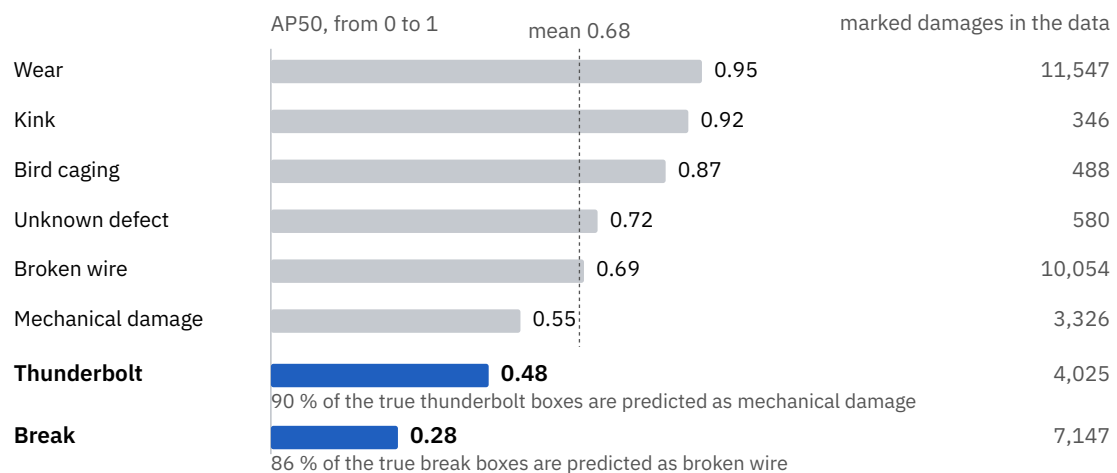
## The neural network

We use YOLO26s, an object detector with 9.5 million parameters that was released in January 2026 for small computers without a graphics card [14]. It draws a box around every damage and gives a score of how sure it is.

For training we merged two public collections of rope photos [15] [16] with a script that unifies their class lists. The merged set has about 22,000 images with 37,513 marked damages in eight classes. We trained for 150 rounds on a graphics card and converted the result to NCNN, a format that runs fast on the ARM processor of the Pi. On the validation photos, which were not used for training, the network finds 96 % of all marked damages, and 74 % of its boxes are real damages.

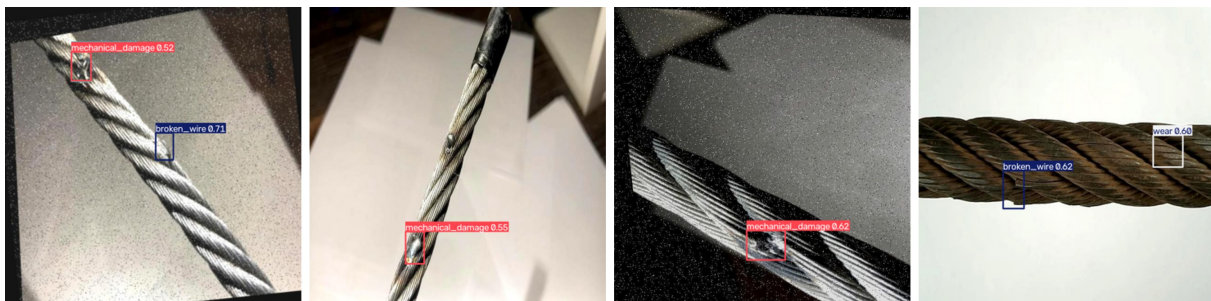
### Mean AP50 is 0.68, the two weakest classes lose their detections to a twin name

Average precision per damage class at IoU 0.5 on the validation set, first training run



**Figure 12.** Validation results of our first training run on 13 September 2026.

Two decisions follow from these numbers. The robot reports a box as soon as the network is 10 % sure. A false alarm costs the operator one look at a photo, and a missed broken wire can cost much more. And it reports each damage in the two groups that rope inspectors know, local fault and loss of metallic area. Figure 12 shows why that helps. The network confuses “break” with “broken wire” and “thunderbolt” with “mechanical damage”, because the two photo collections use different names for the same damage. All four are local faults, so the report of the robot stays correct.



**Figure 13.** Four photos from the public test set with the boxes and confidence values of our network.

The network runs as a separate program. Before that change every analysis froze the whole robot for a few hundred milliseconds, including the thread that feeds the motor.

These numbers come from photos of other people’s ropes. On our own test rope the detection works well in our runs (section 4.6).

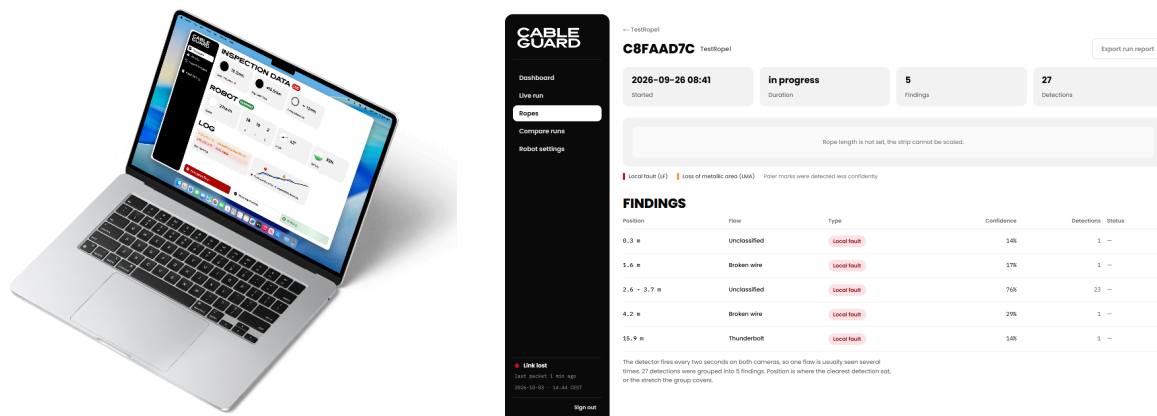
## Passing a rope socket

A rope ends in a thick metal fitting, the rope socket, and the camera ring cannot turn past it. A laser distance sensor looks ahead. While the robot still stands it averages ten readings to learn what the sensor normally sees. When something appears 10 cm nearer than that, the ring parks at the one angle that leaves room for the socket and the network pauses. 50 cm after the last sighting the ring closes and the scan continues. A tower is passed the same way (section 4.3).

## Server and web app

The robot keeps two connections to the server, one for commands and results and one for video, so a video frame cannot delay a stop command. Every result is written to a journal on the memory card before it is sent, and after a connection loss the robot continues at the byte where it stopped. If the connection breaks while the robot drives, it stops and sounds the fault signal, because nobody could stop it remotely.

The web app shows both camera streams, position and speed live, keeps every inspection of a rope and compares two of them. Findings less than 1.5 m apart count as the same damage, so the operator sees what is new and what has disappeared. Operators log in with a password or by holding their phone to an NFC tag on the robot.



**Figure 14.** The web app: the dashboard on a laptop and the findings of one run with position, flaw and confidence.

## What the robot decides by itself

| Decision                                    | What it is based on                               |
|---------------------------------------------|---------------------------------------------------|
| Driving speed                               | Its own timing test at start-up                   |
| Speed of the ring                           | The time the network needs for one pair of photos |
| Open and close for a rope socket or a tower | Distance sensor compared with its own calibration |
| Pause the network                           | Standing still, which keeps the processor cool    |
| Stop                                        | Connection lost while driving                     |
| Accept new settings                         | Only when the drive stands and the ring is parked |

## 3.6 Challenges and tests

Six problems shaped the current design, besides the magnetic head (section 3.1) and the two unseen sides of the rope (section 3.3).

| Problem                   | What we saw                                                           | What we changed                                                 |
|---------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------|
| Wheels slip               | Printed PLA wheels slipped on the steel rope                          | TPU insert with a V-shaped groove that the rope is pressed into |
| Motor stutters            | Pulses from Python came late whenever the network ran                 | Pulses from the PIO hardware block                              |
| Network freezes the robot | Every analysis blocked all other threads                              | Network in its own process                                      |
| Rope is skipped           | The operator could drive faster than the network could analyse        | The robot sets its own speed                                    |
| Photos unusable           | Automatic exposure changed the brightness, the focus was wrong        | Fixed exposure, gain and white balance, focus locked at 5 cm    |
| Position jumps            | New drive settings during a ramp changed the meaning of counted steps | Settings apply only while the robot stands                      |

We test on our own rig (Figure 15), a steel rope of 22 mm diameter and 1.6 m length. There we run complete inspections: start by button, scan with the swinging ring, findings in the web app, stop. Two scripts test the connection to the server from the side of the robot and from the side of the browser, and the detector can be tested alone on a laptop with a webcam.



**Figure 15.** Cableguard on a 22 mm rope like the one on our test rig, seen from the side with the buttons. Rendered from our CAD model.

What is still missing are measurements. We have not yet measured the position error over distance, the hits and misses on prepared damage, the battery runtime, the steepest slope the robot can climb or the behaviour in rain and frost, and we have no automated software tests.

**Two estimates we will check first.** Our test rope is level, but many small cableways are steep. On a slope of 30 degrees the driven wheel has to pull about 15 N ( $3 \text{ kg} \times 9.81 \text{ m/s}^2 \times \sin 30^\circ$ ), while only about half of the weight presses it onto the rope. We will tilt our rig until the wheel slips. If the angle is too low, both wheels will be driven or a spring-loaded roller will press the rope from below. The battery is the second question. The Pi 5 needs about 7 W while the network runs, and two NEMA 23 motors on TB6600 drivers can need 10 to 20 W each, also while they hold still. With 30 to 45 W, a 3S battery of 5,000 mAh (about 55 Wh) would last only one to two hours. We will measure the real consumption with a power meter. If the estimate holds, we will lower the holding current of the ring motor while it stands or use a larger battery.



## 3.7 Use of AI

| AI system               | What we used it for                                              | How much                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
|-------------------------|------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| YOLO26s (Ultralytics)   | Part of the robot. Finds damage in the camera images             | We trained it ourselves on public images and integrated it into the robot                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| Claude Code (Anthropic) | Agentic coding assistant                                         | Used mainly for the web app in a spec-driven workflow: we wrote detailed specifications with acceptance criteria, an agent implemented each change on an isolated feature branch, and every change was merged only through a pull request after human code review. We also used it for debugging and for root-cause analysis of stack traces, logs and race conditions between processes. Hardware-level code such as the PIO stepper driver and the camera ring was written mostly by hand, because it only became reliable through iteration on the real hardware. |
| Claude (Anthropic)      | Grammar check of this report, because we are not native speakers | Claude checked our spelling in English and fixed mistakes and made the text more fluent and readable.                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

## 4 Social impact and innovation

---

### 4.1 Who Cableguard helps

**Operators of small cableways.** Switzerland has about 250 small cableways with a cantonal licence [4]. The control office of the intercantonal concordat (IKSS), to which 21 cantons and Liechtenstein belong, checks them regularly [31], and the safety upgrades it demands can cost an operator up to half a million francs [17]. Cableguard takes over one recurring task, the visual check of the rope, and turns it into data that the operator owns.

**The people who inspect.** Today a person watches the rope by eye from a slow-moving gondola. With Cableguard that person reviews the photos the robot has marked and decides what to do. The judgement stays with a human and the hours of watching go to the robot.

**The heritage.** Central Switzerland, and Nidwalden above all, is seen as the cradle of cableway building and has the highest density of small cableways in the world. About 20 % of the Alps are reached only by these small cableways [4]. One example is the line from Fell to Spis in Nidwalden. It has run since 1932, was first driven by water and leads on to the Spis–Sinsgäu line in Wolfenschiessen, which the cableway builder Niederberger built in 1945 [4]. Lines like these are why we built Cableguard. A rope that is replaced at the right time keeps such an installation running, and every inspection adds photos to its record. This serves UN target 11.4, to “strengthen efforts to protect and safeguard the world’s cultural and natural heritage” [18].

**Passengers.** Rope failures are rare, but each one is a disaster. The deadliest cableway accident in Swiss history happened on 12 July 1972 between Betten and Bettmeralp in Valais: the haul rope broke because of rust, although it had been inspected regularly, and twelve people died [35]. At Mottarone in 2021 a haul rope that fatigue and corrosion had weakened for years broke and 14 people died (section 2) [21] [29]. On 3 September 2025 the cable of the Glória funicular in Lisbon, a national monument opened in 1885, broke and 16 people died [19] [20]. It did not meet the operator’s specifications and was not certified for passenger transport [30]. Three accidents and 42 deaths, and each time the rope was the part that failed.

In none of these cases did the damage appear overnight. Rust and broken wires grow over months and years. A magnetic test by a testing body happens only every few years. In Austria, whose rules follow EN 12927, the limit is three years for a haul rope and up to six years for a track rope [36]. Between two tests the rope is checked by eye, and that check leaves no record that the next one could be compared with. Cableguard works in exactly this gap. It photographs the whole rope as often as the operator wants, and the web app shows which findings are new since the last run. Damage that grows slowly then shows up as a change between two runs.

## 4.2 Possible negative effects

| Risk                                                                                 | What we do about it                                                                                                                                                     |
|--------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| False safety: an operator reads “no findings” as “the rope is healthy”               | Every inspection report will state its limits: surface only, free rope length only. Cableguard adds to the magnetic test by an accredited body and never replaces it    |
| Missed damage: on the validation photos the network misses 4 % of the marked damages | Low reporting threshold, a human reviews every photo, and we retrain with our own photos                                                                                |
| The robot as a hazard: 3 kg hang on a rope above the ground                          | It runs only while the installation is closed. A catch that holds the robot if a wheel leaves the rope is one of our next steps. It can automatically run in the night. |
| Jobs: less inspection work for people                                                | The legally required tests stay with the testing bodies. The work of the operator shifts from watching the rope to judging findings                                     |
| Data: photos and findings of an installation sit on a server                         | Access only with a login. The operator owns the data, and the server can run at the operator’s site                                                                     |

## 4.3 A tried use case

A run on our test rig, a steel rope of 22 mm, looks like this. We turn the camera ring to its open position, hang the robot on the rope and press the red button. The robot times its network, sets its speed and starts to drive. The ring swings back and forth, and both camera pictures appear live in the web app together with the position. Every finding shows up with its photo and its position on the rope. The white button stops the run.



**Figure 16.** The control side of the robot. Two buttons and a display are all an operator needs to start and stop a run. Rendered from our CAD model.

On a real installation the same run would happen after the last ride of the day. By our calculation the robot covers about 110 m per hour, so 800 m of rope take a little more than seven hours. That fits into one night only with a larger battery or a battery change (section 3.6). In the morning the web app lists the findings with photo and position and marks which ones are new, and the operator decides which spots to look at before the first passengers arrive.

Cableguard works in two modes, because a cableway has two kinds of rope.

A **track rope** never moves, so the robot drives along it. A long track rope is often made of several pieces, because it cannot be spliced. The pieces are joined by **rope sockets**: thick metal sleeves into which the wire ends are cast and which are bolted together. A socket is much wider than the rope, so a closed ring could not pass it. Cableguard’s camera ring therefore has an opening of 90 degrees. A laser distance sensor looks ahead, and when something appears 10 cm nearer than the rope it calibrated on, the ring turns its opening towards the obstacle, the network pauses and the robot drives on. 50 cm after the last sighting the ring closes and the scan continues. At a tower the robot uses the same routine: the track rope rests in a rope support that is held by a

single arm from one side, and the arm slides through the opening. One run can therefore cover a track rope along its whole length.

A **haul rope** is an endless loop that runs around the wheels in the stations. Here the robot drives against the direction of the rope, so it stays at the station with only its camera ring working, while the operator runs the rope slowly through it.

## 4.4 Key innovation

Our slogan is **Guarding the lifelines of the Alps.**

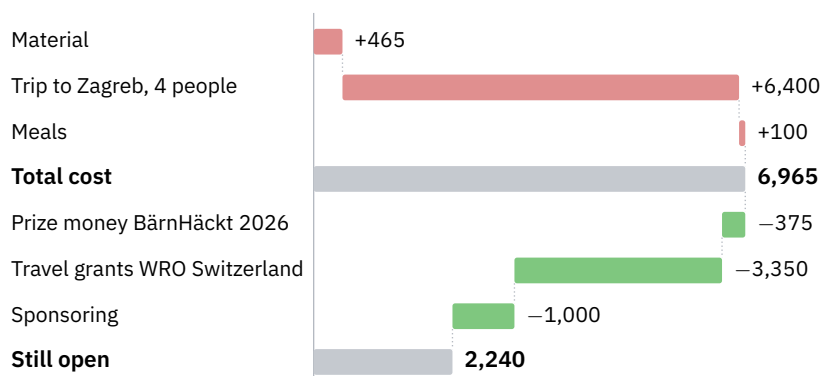
1. **The robot paces itself by its own network.** It measures how fast it can analyse and drives exactly that fast. An operator cannot make it skip rope by driving too fast.
2. **AI on the robot, not in the cloud.** The network runs on the Pi itself, so the analysis works even without a connection.
3. **A rope gets a history.** Every finding has a photo and a position, and two inspections are compared automatically.
4. **The price.** The parts of the prototype cost about CHF 400 (section 4.5).

## 4.5 From prototype to start-up

We may want to turn Cableguard into a company. This section shows what the project has cost, what one robot costs, who would pay for it and what we need to get there.

### CHF 2,240 of our CHF 6,965 budget are still open

Finance plan of the Cableguard project in Swiss francs, as of 2 October 2026



Red bars are costs, green bars are money we have, grey bars are totals.

**Figure 17.** Finance plan of the team on 2 October 2026.

The robot itself is cheap: all material of this season cost CHF 465, including the Hall sensors and magnets of version 2 (Figure 17). We thank the **Lions Club** and WRO Switzerland for their support. We have also contacted rope and ropeway manufacturers and mountain railways, the same companies we need as partners for a pilot.

## Business model on one page

|                                                                                                                                                      |                                                                                                                                                                        |                                                                                                                                                                       |
|------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Customers</b><br>Operators of small cableways in Switzerland, about 250 installations. Later larger ropeways, then cranes, elevators and bridges. | <b>Value</b><br>A rope inspection without a crew, as often as the operator wants. A photo and a position for every finding, and a comparison with the last inspection. | <b>Revenue</b><br><b>Estimated:</b> Inspection as a service at CHF 400 per rope. Subscription at CHF 1,800 per robot and year. Later the magnetic head as an upgrade. |
| <b>Channels</b><br>Direct contact with operators. One pilot installation as our reference.                                                           | <b>Customer relationship</b><br>Web app, model updates and a replacement robot as part of the subscription.                                                            | <b>Costs</b><br>Parts of about CHF 600 per field robot, a SIM card, the server and our time.                                                                          |
| <b>Key resources</b><br>The robot design, about 9,700 lines of software, the trained network and a growing collection of rope photos.                | <b>Key activities</b><br>Build robots, train the network, run inspections and support operators.                                                                       | <b>Key partners</b><br>A pilot operator, a testing body and a workshop for the magnetic head. All three are still to be found.                                        |

The nine fields of the business model canvas for Cableguard. The prices are planning assumptions.

## Cost structure

The parts of one robot cost about CHF 400.

| Parts of one robot                                                                                                           | Cost in CHF |
|------------------------------------------------------------------------------------------------------------------------------|-------------|
| Bought this season: two camera modules with cables, LTE module, distance sensor, filament, steel rod for the axles           | 186.10      |
| From our stock: Raspberry Pi 5, two NEMA 23 motors with drivers, battery, converter, display, buttons, buzzer, belts, screws | about 210   |
| Total                                                                                                                        | about 400   |

A field version with better material, seals and more durable parts would cost about CHF 600 per unit. Its development also costs more. The running costs per robot are a SIM card with a data plan and a share of the server. The largest cost of a company would be our own time, which we do not pay ourselves today.

One more cost is the licence of the network software. Ultralytics YOLO is published under the AGPL-3.0 licence, which is fine for WRO and for open software. A closed commercial product needs the paid Enterprise licence of Ultralytics, which is priced individually by the size of the company and the use case [32]. For a small start-up like ours this should stay affordable. The alternative is to publish our own code under the same AGPL-3.0 licence [32].

## Customers

We start where we live. Switzerland has about 2,450 cableway installations, about 660 with a federal licence and the rest under the cantons [2]. Our first customers are the roughly 250 small cableways, where every second operator has financial problems [4]. The market abroad is large. The Doppelmayr/Garaventa group alone has built more than 15,700 installations in 97 countries [22]. One study values the market for cable cars and ropeways at USD 5.5 billion in 2024 and USD 11.2 billion in 2032, driven by cable cars in cities [23]. Cranes, elevators and bridges use the same ropes and are a later market.

We did the math for our first market ourselves. If each of the 250 small cableways had one rope inspected per year at CHF 400, the market would be about CHF 100,000 a year. That is enough to start, but not enough for a company. We therefore see three circles: the 250 small cableways as the market we can reach first, all 2,450 Swiss installations including the 660 with a federal licence

as the market our product can serve, and ropeways, cranes, elevators and bridges worldwide as the whole market.

Many small cableways have little money, so the operator may not be the one who pays. Cantons, foundations, associations that preserve small cableways and insurers all have an interest in safe ropes. During the pilot we will ask them whether they would pay for inspections or support the purchase of a robot.

### Revenue streams

| Offer                                                                                                      | For whom                              | Price                        |
|------------------------------------------------------------------------------------------------------------|---------------------------------------|------------------------------|
| Inspection as a service. We come with the robot, the operator gets the findings with photos and positions  | Small cableways with one or two ropes | CHF 400 per rope             |
| Subscription. The robot stays with the operator, with web app, model updates and a replacement if it fails | Operators with several installations  | CHF 1,800 per robot and year |

Both prices are planning assumptions. We will set them after the pilot, when we know what an inspection costs an operator today in money and in hours. With these assumptions a subscription pays for a field robot of CHF 600 within four months. It leaves CHF 1,200 in the first year and CHF 1,800 in every year after, which has to pay for the SIM card, the server, support and development. The magnetic head would later be sold as an upgrade.

### Key resources and partners

Our resources are the design of the robot, its electronics and software, the trained network and the team that built all three. A fourth resource grows with every run, because each inspection adds photos of real ropes that make the next network better.

We need three partners.

- An operator who lets us run Cableguard on a real rope for one season.
- A testing body that compares our findings with its magnetic test of the same rope. Without that comparison nobody should trust our numbers.
- A workshop or manufacturer that can machine the iron parts of the magnetic head.

We have talked to several people from the ropeway sector. We are in deep conversations and have been invited to the most used cableway in Switzerland, at the airport, where we will soon ask for a test.

### Company and funding

Today Kirchenfeldrobotics is a registered association, which is enough for a pilot. A limited company (GmbH) needs CHF 20,000 of share capital plus about CHF 2,000 to 4,600 for advice, the notary and the commercial register [24]. We plan to found it when the first operator pays, and until then we finance the pilot with prize money, sponsors and foundations. Our first stop is the free consultation of be-advanced, the innovation agency of the Canton of Bern [25]. Later we want to apply for the Initial Coaching of Innosuisse, a voucher of up to CHF 10,000 for start-up coaching [26].

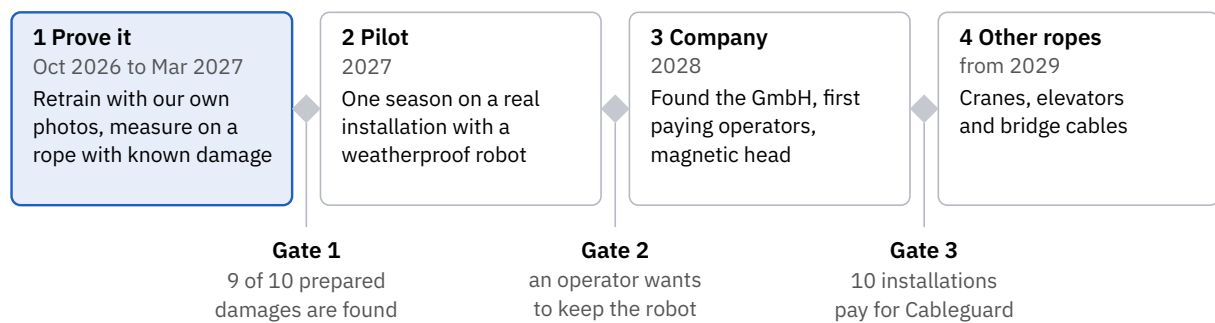
## 4.6 Next steps

The prototype works on our test rig. Five steps separate it from a robot that an operator can use (Figure 18).

1. **Real numbers and better eyes.** We count hits, misses and false alarms on a rope with prepared damage. Then we merge the twin classes of the two photo collections and retrain the network



## Each step towards a company has a gate that Cableguard has to pass



**Figure 18.** Our roadmap: four phases and three gates.

with photos from our own cameras. The photos the robot stores with every finding are the start of that collection.

2. **More speed.** At about 110 m per hour a long rope takes more than one night. The network sets the speed, so we will test a smaller network, an AI accelerator for the Raspberry Pi and a longer photo.
3. **Ready for weather.** A casing of ASA or aluminium with sealed openings, tested in rain and frost, and a catch that holds the robot if a wheel leaves the rope.
4. **Inside the rope.** We have the ring of Hall sensors and a software interface for a second sensor. Missing are the magnets in a yoke machined from iron, a safe way to put this head on the rope and a field model that turns the readings into findings. We want to solve this with a partner who has the machines.
5. **A report an inspector can file.** Each run should end in a PDF with every finding, its photo and position, a rating of its severity and a clear statement of what was not inspected.

For the first gate we need no partner, only a rope with prepared damage on our rig. For the second we need one operator and one rope, ideally a historic line like Fell–Spis in Nidwalden, which has run since 1932 [4]. Small cableways like this reach about 20 % of the Alps, and every second one has financial problems [4], so a cheap and frequent rope check can decide whether a line stays open. That is what we mean by guarding the lifelines of the Alps.

# Sources

---

## Team and problem

1. <https://kirchenfeldrobotics.ch>
2. [https://efk.admin.ch/wp-content/uploads/publikationen/berichte/sicherheit\\_und\\_umwelt/verkehr\\_und\\_umwelt/23767/23767be-wik-d.pdf](https://efk.admin.ch/wp-content/uploads/publikationen/berichte/sicherheit_und_umwelt/verkehr_und_umwelt/23767/23767be-wik-d.pdf)
3. [https://en.wikipedia.org/wiki/Schweizer\\_Seilbahninventar](https://en.wikipedia.org/wiki/Schweizer_Seilbahninventar) and <https://www.seilbahninventar.ch>
4. <https://www.srf.ch/radio-srf-1/im-ueberlebenskampf-kleinseilbahnen-kulturgut-in-luftiger-schieflage>
5. <https://www.ift.uni-stuttgart.de/institut/pdfs/Seilbahn-Broschuere-IFT.pdf>
6. <https://pmc.ncbi.nlm.nih.gov/articles/PMC5795951/>
7. <https://wro-association.org/wp-content/uploads/WRO-2026-Future-Innovators-Mission.pdf>

## Robotic solution

8. <https://doi.org/10.3390/ma15207362>
9. <https://patents.google.com/patent/US8718352>
10. <https://inspenet.com/en/news/intelligent-robot-improves-bridge-inspection-with-machine-vision/>
11. <https://www.raspberrypi.com/products/raspberry-pi-5/>
12. <https://www.filamentive.com/outdoor-3d-printing-2025-uv-resistant-filament-uk-guide/>
13. <https://www.raspberrypi.com/news/piolib-a-userspace-library-for-pio-control/>
14. <https://docs.ultralytics.com/models/yolo26/>
15. <https://universe.roboflow.com/wirerope-qfgck/wire-rope-defect-f4qyg>
16. <https://universe.roboflow.com/yuan-9voxc/wire-rope-defect-review>

## Social impact and start-up

17. <https://www.beobachter.ch/gesellschaft/kleinseilbahnen-nicht-sicher-vielen-seilbahnli-droht-das-aus>
18. <https://sdgs.un.org/goals/goal11>
19. [https://en.wikipedia.org/wiki/2025\\_Ascensor\\_da\\_Gl%C3%B3ria\\_derailment](https://en.wikipedia.org/wiki/2025_Ascensor_da_Gl%C3%B3ria_derailment)
20. [https://en.wikipedia.org/wiki/Ascensor\\_da\\_Gl%C3%B3ria](https://en.wikipedia.org/wiki/Ascensor_da_Gl%C3%B3ria)
21. [https://en.wikipedia.org/wiki/Stresa%E2%80%93Mottarone\\_cable\\_car\\_crash](https://en.wikipedia.org/wiki/Stresa%E2%80%93Mottarone_cable_car_crash)
22. [https://en.wikipedia.org/wiki/Doppelmayr/Garaventa\\_Group](https://en.wikipedia.org/wiki/Doppelmayr/Garaventa_Group)
23. <https://www.credenceresearch.com/report/cable-car-and-ropeways-market>
24. <https://www.kmu.admin.ch/de/gmbh-haftung-stammkapital-gruendung>
25. <https://www.berninvest.be.ch/en/start/dienstleistungen/coaching-angebot.html>
26. <https://www.innosuisse.admin.ch/en/initial-coaching-for-start-up>

## Further sources

Sources 27 to 36 were opened on 3 October 2026.

27. [https://oitaf.org/wp-content/uploads/2023/12/501814\\_Referat\\_Winter\\_-\\_Kuehner\\_-\\_Pernot.pdf](https://oitaf.org/wp-content/uploads/2023/12/501814_Referat_Winter_-_Kuehner_-_Pernot.pdf)
28. EN 12927:2019, Safety requirements for cableway installations designed to carry persons, Ropes (CEN)
29. <https://www.swissinfo.ch/ita/italia-mottarone-perizia-68-fune-gi%C3%A0-rotta-prima-tragedia/47908270>
30. <https://econews.pt/?p=45601>
31. <https://www.zh.ch/de/mobilitaet/transportbewilligungen/personentransporte/bewilligung-kleinseilbahnen-skilifte.html>
32. <https://www.ultralytics.com/license>
33. <https://www.srf.ch/news/schweiz/neue-spitzenwerte-permafrost-taut-immer-weiter-auf>
34. [https://www.bav.admin.ch/dam/de/sd-web/rtTulUXa8A3o/leitfaden\\_permafrost.pdf](https://www.bav.admin.ch/dam/de/sd-web/rtTulUXa8A3o/leitfaden_permafrost.pdf)
35. <https://www.srf.ch/news/schweiz/unglueck-vor-50-jahren-bettmeralp-das-sind-die-lehren-aus-der-seilbahn-katastrophe>
36. <https://www.wko.at/oe/transport-verkehr/seilbahnen/richtlinie-aenderung-intervall-augenscheinuntersuchung.pdf>